

Calculatrice C

Code Source

Réalisé par : LIMOUR Oumaima

BAKKALI Dina

EL ALOUY EL AMERANY Aya

Encadrant : EL ABDELKHALKI Jamal

calculatrice.c

```
#include <gtk/gtk.h>
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <string.h>

typedef struct {
    GtkWidget *display;
    double a;
    char op;
    gboolean reset;
} Calc;

/* ----- DISPLAY ----- */
void set_text(Calc *c, const char *t) {
    gtk_editable_set_text(GTK_EDITABLE(c->display), t);
}

/* FIX IMPORTANT : lecture fiable */
double get_val(Calc *c) {
    const char *t = gtk_editable_get_text(GTK_EDITABLE(c->display));

    if (!t || strlen(t) == 0)
        return 0;

    char buffer[100];
    strncpy(buffer, t, sizeof(buffer));
    buffer[sizeof(buffer) - 1] = '\0';

    return strtod(buffer, NULL);
}

/* ----- DIGIT (FIX DECIMAL COMPLET) ----- */
void digit(GtkButton *b, gpointer d) {
    Calc *c = d;
    const char *v = gtk_button_get_label(b);
    const char *cur = gtk_editable_get_text(GTK_EDITABLE(c->display));

    /* gestion point decimal correcte */
    if (!strcmp(v, ".")) {
        if (c->reset) {
            set_text(c, "0.");
        }
    }
}
```

```

        c->reset = FALSE;
        return;
    }
    if (strchr(cur, '.') != NULL)
        return;
}

if (c->reset || strcmp(cur, "0") == 0) {
    set_text(c, v);
    c->reset = FALSE;
} else {
    char buf[100];
    snprintf(buf, sizeof(buf), "%s%s", cur, v);
    set_text(c, buf);
}
}

/* ----- OP ----- */
void op(GtkButton *b, gpointer d) {
    Calc *c = d;

    const char *txt = gtk_editable_get_text(GTK_EDITABLE(c->display));
    c->a = strtod(txt, NULL);
    c->op = gtk_button_get_label(b)[0];
    c->reset = TRUE;
}

/* ----- CLEAR ----- */
void clear_all(GtkButton *b, gpointer d) {
    Calc *c = d;
    set_text(c, "0");
    c->a = 0;
    c->op = 0;
    c->reset = TRUE;
}

/* ----- BACKSPACE ----- */
void backspace(GtkButton *b, gpointer d) {
    Calc *c = d;

    char buf[100];
    strcpy(buf, gtk_editable_get_text(GTK_EDITABLE(c->display)));

    int len = strlen(buf);

    if (len <= 1) {
        set_text(c, "0");
        c->reset = TRUE;
    } else {
        buf[len - 1] = '\0';
        set_text(c, buf);
    }
}

/* ----- SQRT ----- */
void root(GtkButton *b, gpointer d) {
    Calc *c = d;
    double x = get_val(c);

    if (x < 0) {
        set_text(c, "Error");
        c->reset = TRUE;
        return;
    }
}

```

```

    }

    char buf[100];
    snprintf(buf, sizeof(buf), "%.10g", sqrt(x));
    set_text(c, buf);
    c->reset = TRUE;
}

/* ----- EQUAL ----- */
void equal(GtkButton *b, gpointer d) {
    Calc *c = d;

    const char *txt = gtk_editable_get_text(GTK_EDITABLE(c->display));
    double b2 = strtod(txt, NULL);

    double r = 0;

    switch (c->op) {
        case '+': r = c->a + b2; break;
        case '-': r = c->a - b2; break;
        case '*': r = c->a * b2; break;

        case '/':
            if (b2 == 0) {
                set_text(c, "Error");
                return;
            }
            r = c->a / b2;
            break;

        case '%':
            r = c->a * b2 / 100.0;
            break;

        case '^':
            r = pow(c->a, b2);
            break;

        default:
            return;
    }

    char buf[100];
    snprintf(buf, sizeof(buf), "%.10g", r);
    set_text(c, buf);

    c->reset = TRUE;
    c->op = 0;
}

/* ----- BUTTON ----- */
GtkWidget* btn(const char *t, GCallback cb, Calc *c, const char *cls) {
    GtkWidget *b = gtk_button_new_with_label(t);
    g_signal_connect(b, "clicked", cb, c);
    gtk_widget_add_css_class(b, cls);
    return b;
}

/* ----- CSS ----- */
void css() {
    GtkCssProvider *p = gtk_css_provider_new();

```

```

const char *style =
    "window { background:#05060a; }"
    "entry { font-size:64px; padding:30px; background:#05060a; color:#00c8ff;"
border:none; text-align:right; }"
    "button { font-size:26px; min-width:85px; min-height:85px; border-radius:28px;"
margin:6px; }"
    "button: hover { background:#0a1a2a; box-shadow:0 0 18px #00c8ff; }"
    ".digit { background:#0a0f18; color:#e6f7ff; }"
    ".op { background:#0a2238; color:#00c8ff; font-weight:bold; }"
    ".equal { background:#00c8ff; color:#001018; font-weight:bold; }"
    ".clear { background:#1a0a0a; color:#ff4d4d; }"
    ".del { background:#0f1f33; color:#00c8ff; }"
    ".special { background:#0b1a2a; color:#00c8ff; }";

gtk_css_provider_load_from_data(p, style, -1);

gtk_style_context_add_provider_for_display(
    gdk_display_get_default(),
    GTK_STYLE_PROVIDER(p),
    GTK_STYLE_PROVIDER_PRIORITY_APPLICATION
);
}

/* ----- APP ----- */
static void activate(GtkApplication *app, gpointer u) {

    css();

    Calc *c = g_malloc(sizeof(Calc));
    c->a = 0;
    c->op = 0;
    c->reset = TRUE;

    GtkWidget *win = gtk_application_window_new(app);
    gtk_window_set_title(GTK_WINDOW(win), "Calculatrice");
    gtk_window_set_default_size(GTK_WINDOW(win), 420, 720);

    GtkWidget *grid = gtk_grid_new();
    gtk_grid_set_row_spacing(GTK_GRID(grid), 12);
    gtk_grid_set_column_spacing(GTK_GRID(grid), 12);
    gtk_window_set_child(GTK_WINDOW(win), grid);

    /* DISPLAY */
    c->display = gtk_entry_new();
    gtk_editable_set_editable(GTK_EDITABLE(c->display), FALSE);
    set_text(c, "0");
    gtk_grid_attach(GTK_GRID(grid), c->display, 0, 0, 4, 1);

    /* BUTTONS */
    const char *b[5][4] = {
        {"C", "<BS>", "%", "/"},
        {"7", "8", "9", "*"},
        {"4", "5", "6", "-"},
        {"1", "2", "3", "+"},
        {"0", ".", "sqrt", "="}
    };

    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 4; j++) {

            const char *t = b[i][j];
            GtkWidget *button;

```

```

        if (g_ascii_isdigit(t[0]) || !strcmp(t, "."))
            button = btn(t, G_CALLBACK(digit), c, "digit");
        else if (!strcmp(t, "="))
            button = btn(t, G_CALLBACK(equal), c, "equal");
        else if (!strcmp(t, "C"))
            button = btn(t, G_CALLBACK(clear_all), c, "clear");
        else if (!strcmp(t, "<BS>"))
            button = btn(t, G_CALLBACK(backspace), c, "del");
        else if (!strcmp(t, "sqrt"))
            button = btn(t, G_CALLBACK(root), c, "special");
        else
            button = btn(t, G_CALLBACK(op), c, "op");

        gtk_grid_attach(GTK_GRID(grid), button, j, i + 1, 1, 1);
    }
}

gtk_window_present(GTK_WINDOW(win));
}

/* ----- MAIN ----- */
int main(int argc, char **argv) {

    GtkApplication *app = gtk_application_new(
        "org.modern.calc",
        G_APPLICATION_DEFAULT_FLAGS
    );

    g_signal_connect(app, "activate", G_CALLBACK(activate), NULL);

    return g_application_run(G_APPLICATION(app), argc, argv);
}

```